



CH

Ingesting and Preparing Data

- ☐ Introduction : AWS
- ☐ Data pipeline
- ☐ Data Sources
- ☐ Data Ingestion
- ☐ ETL & ELT
 - Wrangling
 - Discovery
 - Structuring
 - Cleaning
 - Enriching
 - Validating
 - Publishing

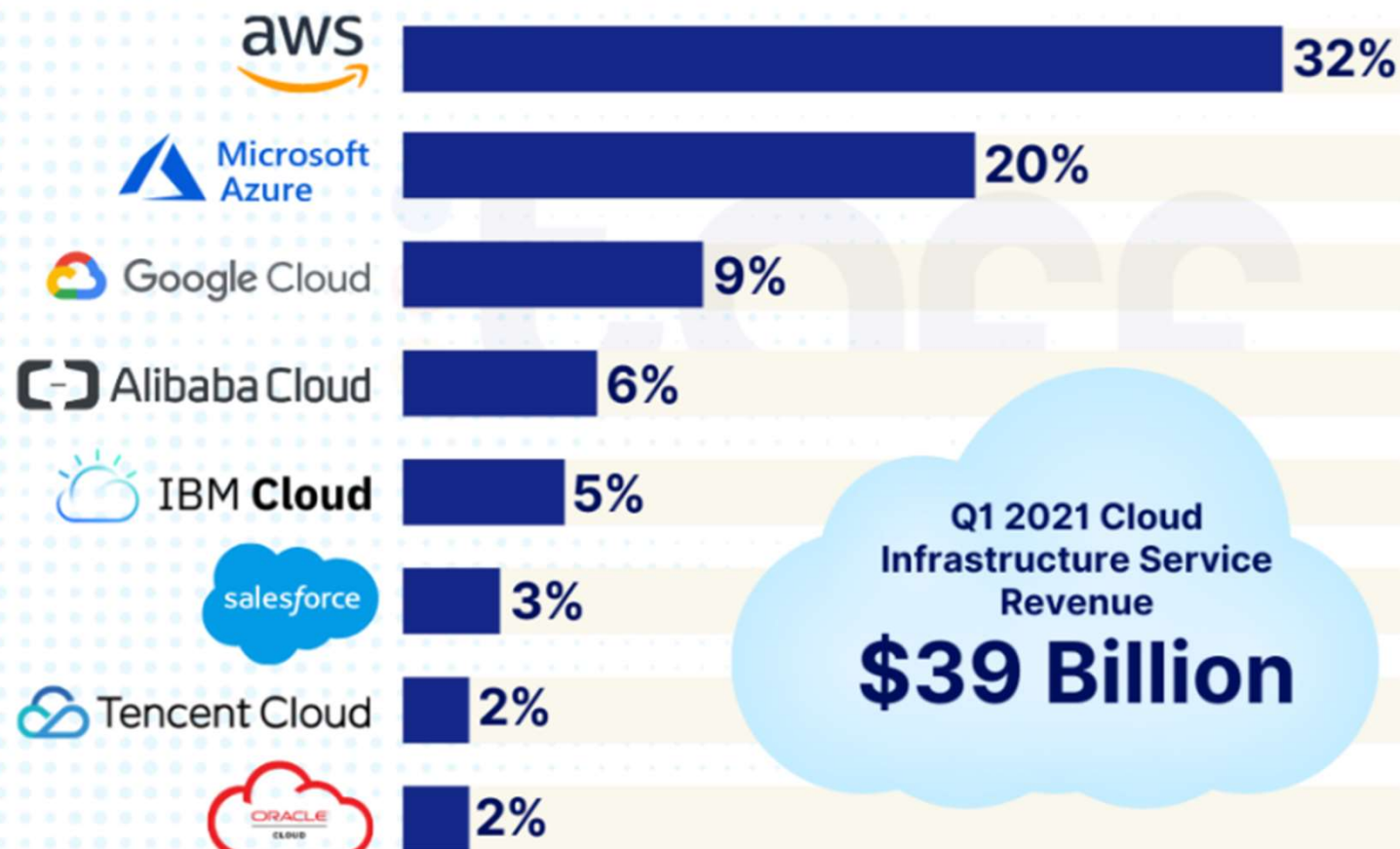


Anis Ben Aicha: AWS academic instructor



Amazon Leads \$150-Billion Cloud Market

Worldwide market share of leading cloud infrastructure service providers in Q1 2021*





Cloud computing with AWS

- Amazon Web Services (AWS) is the world's most comprehensive and widely adopted cloud platform. It offers more than **200** comprehensive services from data centres around the world.
- Millions of customers use AWS to reduce costs, increase agility and innovate faster.
- AWS has many services and features: infrastructure technologies such as **compute**, **storage** and **databases**, to emerging technologies such as **machine learning** and **artificial intelligence**, **data lakes** and **analytics** and the Internet of Things → moving applications to the cloud and creating almost anything is **faster, easier and more cost-effective**.
- AWS also has the deepest functionality within these services. For example, AWS offers the widest variety of databases specifically designed for different types of applications → choose the right tool for the job, to get the best cost and performance.



▪ Solution proposed by AWS (by Use Case)

- Archiving

- Affordable solutions for data archiving from gigabytes to petabytes

- Backup and Restore

- Durable, cost-effective options for backup and disaster recovery

- Blockchain

- Shared ledgers for trusted transactions among multiple parties

- Cloud Migration

- Easily migrate apps and data to AWS

- Cloud Operations

- Operate securely and safely in the cloud, at scale

- Containers

- Fully managed services for every workload

- Content Delivery

- Accelerate websites, APIs, and video content

- Database Migrations

- Save time and cost by migrating to fully managed databases

- Data Lakes and Analytics

- Comprehensive, secure, scalable, and cost-effective data lake and analytics solutions



- DevOps
Rapidly and reliably build and deliver products using DevOps practices
- E-Commerce
Drive small or large e-commerce businesses with our secure and highly scalable solutions for online sales and retail
- Edge Computing
Move data processing and analysis as close to the end user as necessary
- Front-End Web & Mobile Development
Build and deploy secure, scalable mobile and web apps fast
- High Performance Computing
Enhanced networking and cloud-scale clusters for complex problems
- Hybrid Cloud Architectures
Extend your IT infrastructure to the AWS cloud
- Internet of Things
Easily scale to billions of devices and trillions of messages



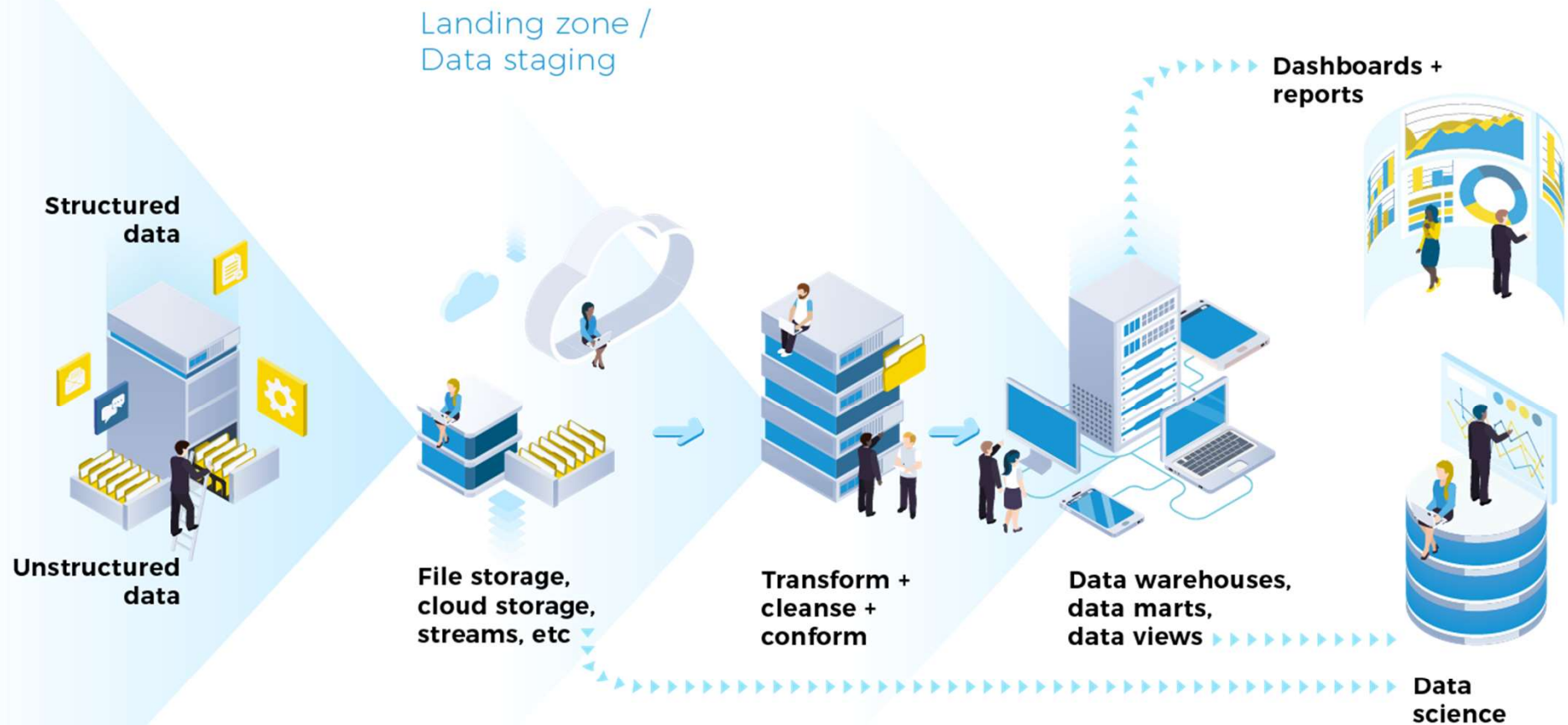
- **Machine Learning**
Build with powerful services and platforms, and the broadest machine learning framework support anywhere
- **Modern Application Development**
Develop and evolve applications through cycles of rapid innovation
- **Multicloud**
Simplify and centralize operations in hybrid and multicloud environments
- **Remote Work**
AWS solutions for remote employees, contact center agents, and creative professionals
- **Resilience**
Build and run resilient, highly available applications
- **Scientific Computing**
Analyze, store, and share massive data sets
- **Serverless Computing**
Build and run applications without thinking about servers
- **Websites**
Reliable, highly scalable, and low cost website and web application hosting



- **The AWS Certified Data Engineer - Associate (DEA-C01)**
 - validates a candidate's ability to implement data pipelines and to monitor, troubleshoot, and optimize cost and performance issues in accordance with best practices.
- The exam also validates a candidate's ability to complete the following tasks:
 - **Ingest and transform data**, and orchestrate data pipelines while applying programming concepts.
 - Choose an optimal data store, design data models, catalog data schemas, and manage data lifecycles.
 - Operationalize, maintain, and monitor data pipelines. Analyze data and ensure data quality.
 - Implement appropriate authentication, authorization, data encryption, privacy, and governance. Enable logging.



Data Pipeline





■ Data Pipeline

- A data pipeline is a **series of processing steps** to **prepare** enterprise data for **analysis**. Organizations have a large volume of data from various sources like applications, Internet of Things (IoT) devices, and other digital channels. However, raw data is useless; it must be moved, sorted, filtered, reformatted, and analyzed for business intelligence.
- A data pipeline **includes various technologies** to verify, summarize, and find patterns in data to inform business decisions.
- Well-organized data pipelines **support various big data projects**, such as data visualizations, exploratory data analyses, and machine learning tasks.

Data Pipeline

Benefits



- **Data Pipeline Benefits:** Data pipelines integrate data from different sources and transform it for analysis. They **remove data silos** and make **data analytics** more **reliable and accurate**.
 - **Improved data quality:** Data pipelines clean and refine raw data, improving its usefulness for end users. They standardize formats for fields like dates and phone numbers while checking for input errors. They also remove redundancy and ensure consistent data quality across the organization.
 - **Efficient data processing:** Data engineers have to perform many repetitive tasks while transforming and loading data. Data pipelines allow them to automate data transformation tasks and focus instead on finding the best business insights. Data pipelines also help data engineers more quickly process raw data that loses value over time.
 - **Comprehensive data integration:** A data pipeline abstracts data transformation functions to integrate data sets from disparate sources. It can cross-check values of the same data from multiple sources and fix inconsistencies. For example, imagine that the same customer makes a purchase from your ecommerce platform and your digital service. However, they misspell their name in the digital service. The pipeline can fix this inconsistency before sending the data for analytics.



Pipeline is created for data analytics purposes and has:

- **Data sources** – These can be internal or external and may be structured (e.g., the result of a database call), semi-structured (e.g., a CSV file or a Google Sheets file), or unstructured (e.g., text documents or images).
- **Ingestion process** – This is the means by which data is moved from the source into the pipeline (e.g., API call, secure file transfer).
- **Transformations** – In most cases, data needs to be transformed from the input format of the raw data, to the one in which it is stored. There may be several transformations in a pipeline.
- **Data quality/cleansing** – Data is checked for quality at various points in the pipeline. Data quality will typically include at least validation of data types and format, as well as conforming with the master data.
- **Enrichment** – Data items may be enriched by adding additional fields, such as reference data.
- **Storage** – Data is stored at various points in the pipeline, usually at least the landing zone and a structured store (such as a data warehouse).



■ Stream processing pipelines

- A data stream is a continuous, incremental sequence of small-sized data packets. It usually represents a series of **events** occurring over a given period. For example, a data stream could show sensor data containing measurements over the last hour.
- A single action, like a financial transaction, can also be called an *event*. Streaming pipelines process a series of events for real-time analytics.
- Streaming data requires low latency and high fault tolerance.
- Data pipeline should be able to process data even if some data packets are lost or arrive in a different order than expected.

■ Batch processing pipelines

- Batch processing data pipelines process and store data in large volumes or batches. They are suitable for occasional high-volume tasks like monthly accounting.
- The data pipeline contains a series of sequenced commands, and every command is run on the entire batch of data. The data pipeline gives the output of one command as the input to the following command. After all data transformations are complete, the pipeline loads the entire batch into a cloud data warehouse or another similar data store.

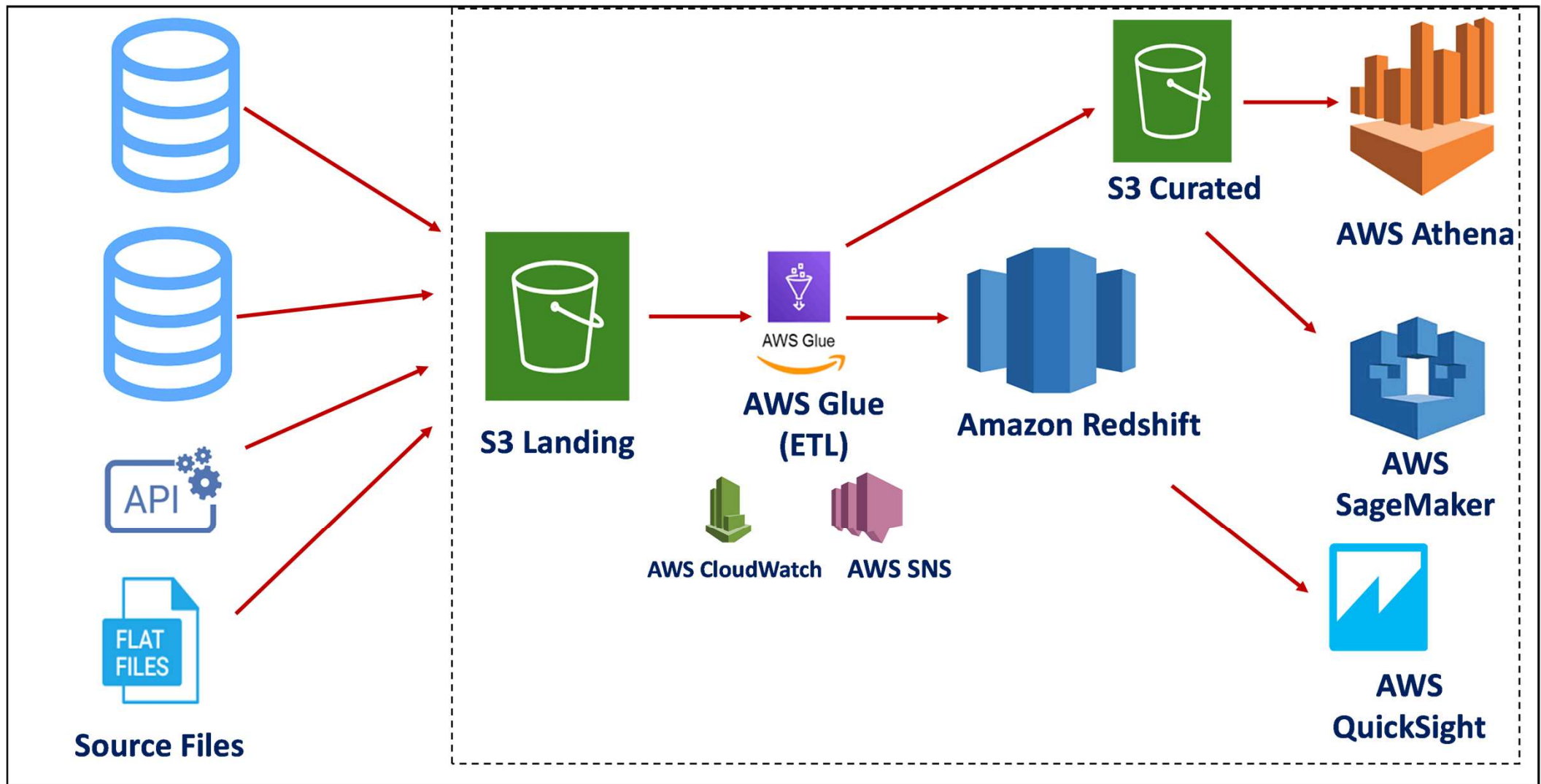


Difference between batch and streaming data pipelines

Batch processing pipelines run infrequently and typically during off-peak hours. They require high computing power for a short period when they run. In contrast, stream processing pipelines run continuously but require low computing power. Instead, they need reliable, low-latency network connections.

Data Pipeline

AWS solution: example





- **S3 bucket** as a landing area for heterogeneous data sources.
- **AWS Glue** is used as an ETL tool, and its related services like AWS CloudWatch for monitoring and security, and AWS SNS for notifications on the status of jobs are used.
- **Amazon Redshift** is a fully managed data warehousing service that uses a columnar storage format and a massively parallel processing (MPP) architecture. It is optimized for high-performance analytics on large datasets.
- **Amazon Athena** is an interactive query service. It allows users to analyze data stored in Amazon S3 using standard SQL queries.
- **Amazon SageMaker** is a fully managed service provided that enables high-performance and low-cost machine learning (ML). It brings together a comprehensive set of tools, such as notebooks, debuggers, profilers, pipelines, and MLOps, in an integrated development environment (IDE) to build, train, and deploy ML models at scale
- **Amazon QuickSight** is a business intelligence (BI) service. It is designed to help users create interactive visualizations, perform ad-hoc analysis, and gain insights from their data.



- Definition of data sources
 - A data source is the place where the data used comes from.
 - A data source can be a database, a flat file, measurements taken directly from physical devices, data obtained by web scraping or one of the many static and streaming data services that abound on the Internet.

- Types of data sources
 - Although the **diversity of data** content, formats and locations continues to increase with the advent of technologies such as IoT and the adoption of big data methodologies → it is possible to classify most data sources into two broad categories:
 - **machine data sources**
 - **file data sources.**



- **Machine Sources:** Machine sources refer to data that is generated or collected automatically by machines or systems. These sources often produce large volumes of data in real-time or near real-time. Examples of machine sources include:
 1. **Sensor data:** Data collected from IoT devices, sensors, or monitoring systems.
 2. **Log files:** Data generated by software applications, servers, or network devices, capturing events, errors, or system activities.
 3. **Streaming data:** Continuous data streams from sources like social media feeds, clickstreams, or financial market data.
- **File Sources:** File sources involve data that is stored in files or documents. These sources typically require manual creation, editing, or uploading of data. Examples of file sources include:
 1. **Databases:** Structured data stored in relational or non-relational databases.
 2. **Flat files:** Text files, spreadsheets, CSV files, XML files, or JSON files.
 3. **Documents:** PDFs, Word documents, presentations, or other file formats containing textual or multimedia content.

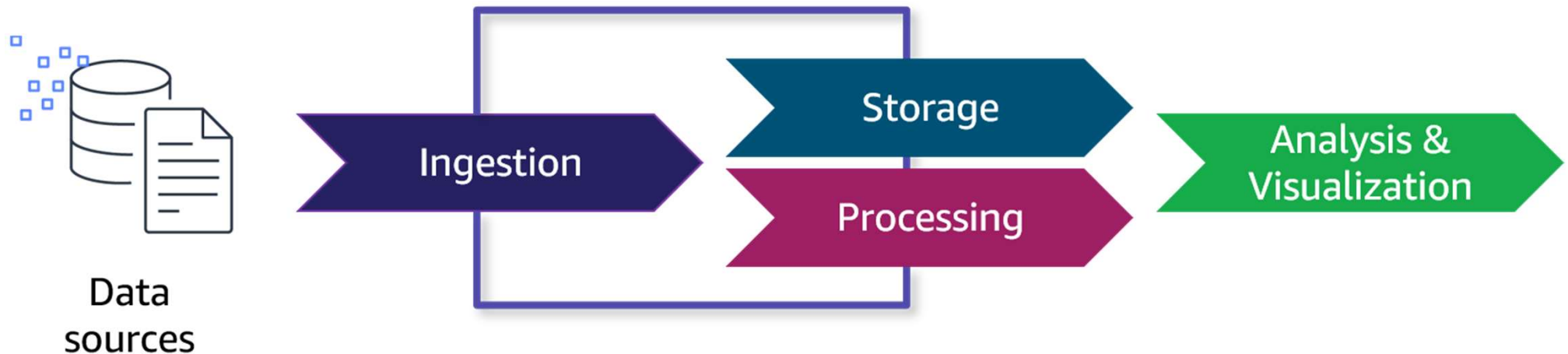
Data ingestion



- Data ingestion refers to the process of collecting data from different sources and centralizing it on a destination site where it is ready to be analyzed and used.
 - Data ingestion makes it possible to extract information from various sources that do not necessarily communicate with each other. By linking them up and centralizing them in a unified system → organization will be able to exploit them → detect previously unknown patterns and correlations.
 - Data ingestion → understanding “**connector**” between various sources of information and the destination site for the business”.
 - Destination sites : database, data warehouse (or data mart), data lake
 - The different types of data ingestion
 - Real-time or streaming ingestion
 - Batch ingestion
 - Lambda and Kappa architectures: combining the Real-time and Batch solutions
- ➔ It is up to each organization to clearly define its data requirements and to choose the right technique according to its business objectives.



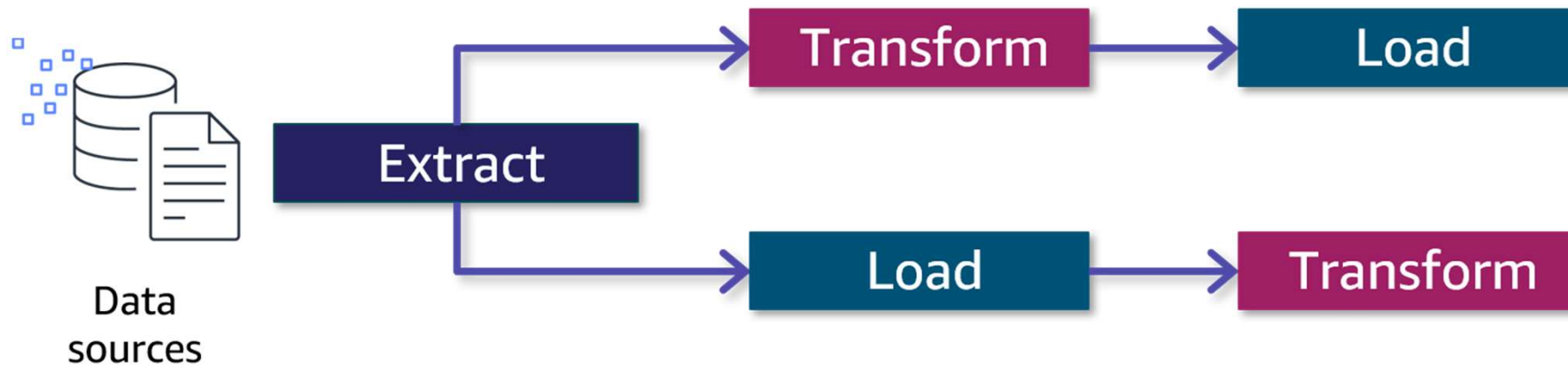
- the ingestion process incorporates extracting data from a source that isn't already available in the pipeline, loading the data into some type of temporary storage in the pipeline, and processing the data by transforming it before or after the data is loaded into more permanent storage.
- The type and scope of transformations, and where they occur across the data pipeline, highly depend on the business need and the nature of the data that is being ingested





Extract transform load (ETL)

1. **Extract** structured data.
2. **Transform** the data into a format that matches the destination.
3. **Load** the data into structured storage for defined analytics.



Extract load transform (ELT)

1. **Extract** unstructured or structured data.
2. **Load** the data into the data lake in the format that is as close to the raw form as possible.
3. **Transform** the data as needed for analytic scenarios.

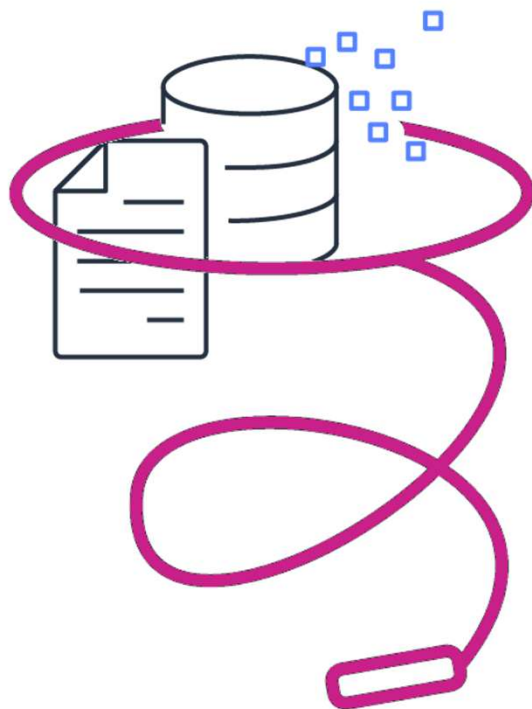


▪ ETL

- The traditional method of ingesting and preparing data for analytics was ETL. With ETL, data is *extracted* from its source, *transformed* into a structured format that is ready to be used in analytics applications, and *loaded* into structured storage, such as a data warehouse. Data scientists or analysts query the data warehouse and might perform additional transformations and processing as part of their analyses.
- Ingestion by using an ETL flow works well with structured data that is destined for a data warehouse or other structured store.

▪ ELT

- The trend towards collecting high volumes of unstructured and semistructured data and the desire to make that data available for a variety of uses led to a different approach for data ingestion: ELT. With an ELT approach, data is extracted from its source and transformed only enough to store it in a consumable but relatively raw state in a data lake. Transformations that might be needed for specific uses are done later, as consumers access data and build analytics solutions with it.
- Ingestion by using an ELT flow works well for unstructured data that is destined for a data lake for a variety of use cases.

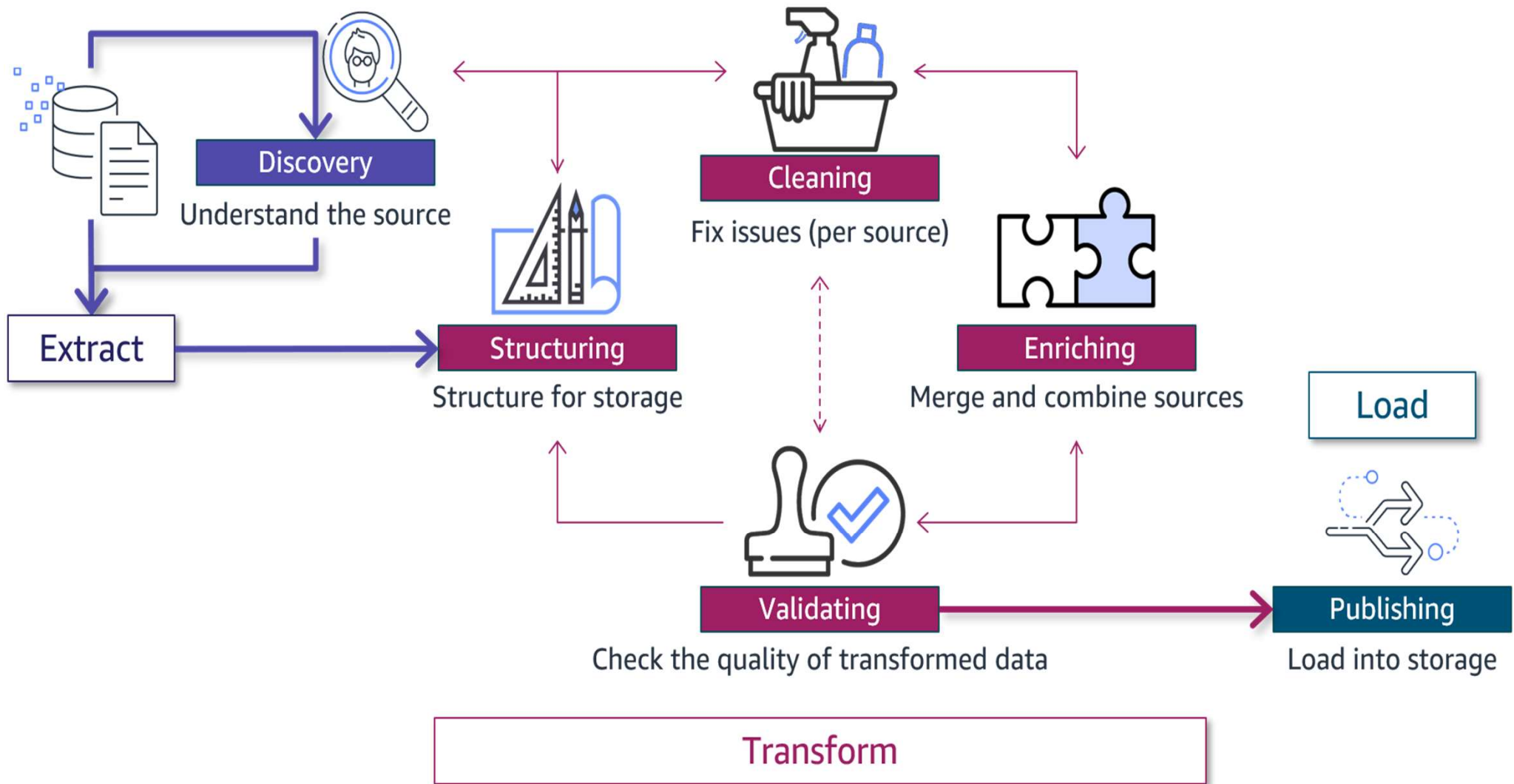


Transforming large amounts of unstructured or structured raw data from multiple sources with different schemas into a meaningful set of data that has value for downstream processes or users

The term *data wrangling* reflects the complexity and messiness of transforming large amounts of unstructured data or sets of structured data with multiple schemas across multiple sources into a meaningful set of data that has value for downstream processes or users.

ETL & ELT

Wrangling: steps

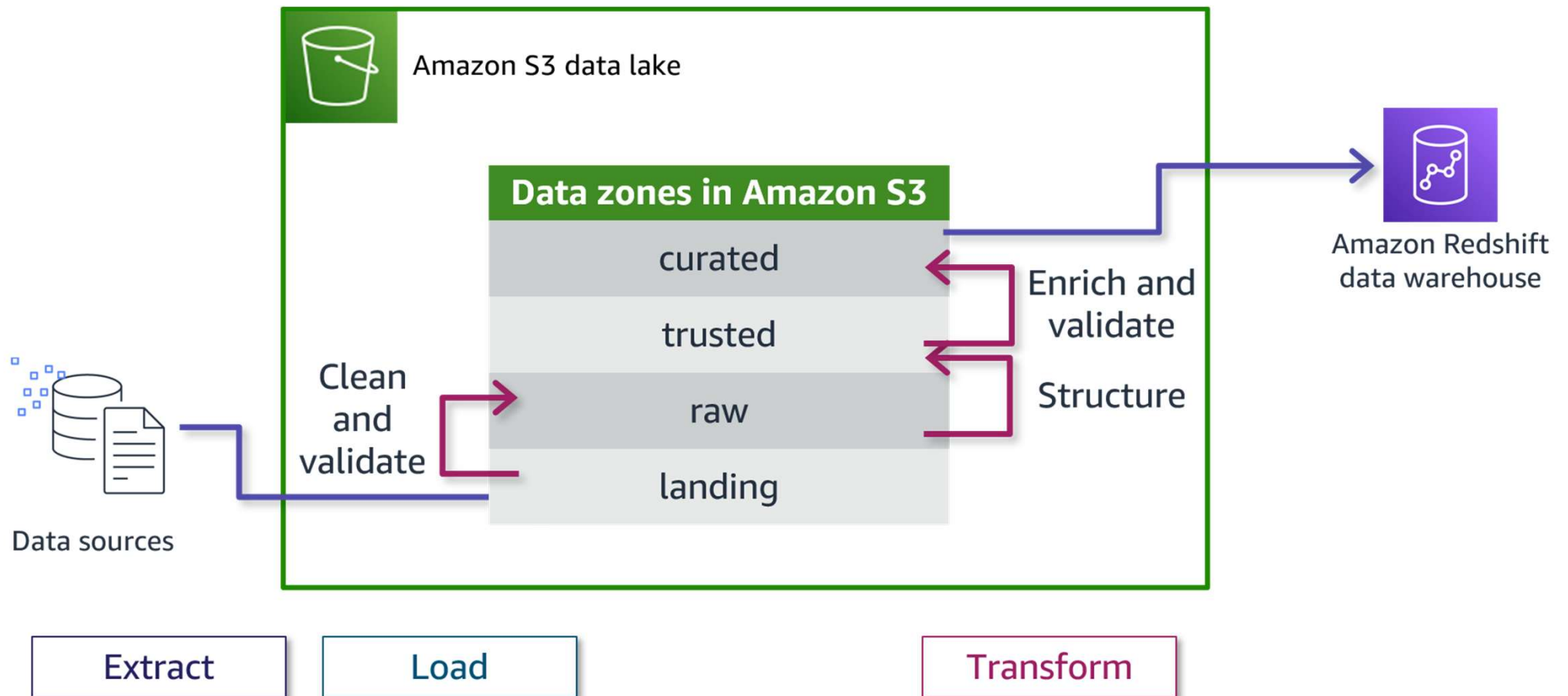




- In practice, there might not be clear delineations between the steps, there might be iterations within and across steps
 - Individual steps might not occur as part of ingesting a data source into the pipeline.
 - Each step might be more or less complex, depending on the goal of the specific ingestion that you are working on
-
- **Data wrangling & ETL & ELT**
 - ETL processing has been more of the data engineer's domain. ETL processing encompasses scripting and automating transformation steps with batch jobs that run on infrastructure that is controlled by IT. Analysts and data scientists worked only with data that was in a defined, consumable state.
 - Data wrangling is more associated with an ELT flow where business users or data scientists transform datasets that are available to them within the pipeline storage layer but that haven't been refined for a specific use case.

ETL & ELT

Example: ELT and data wrangling in a modern data architecture





- Each pipeline's architecture might look **quite different** depending on the **business problem** being addressed, **type of data** being ingested, and skills and **expertise of the team** that supports the pipeline and accesses the data.
- The diagram illustrates an ELT flow and highlights how the data wrangling steps might occur. In this reference architecture, data is ingested into a storage layer that includes an Amazon S3 data lake and an Amazon Redshift data warehouse. Within the data lake, S3 buckets are used as named zones to store data in different states.
- The **landing zone bucket** serves as temporary storage from which you perform just enough cleaning and validating to permanently save the ingested data into the **raw** zone bucket.
- In this example, the transform part of ELT occurs when data in the raw zone of the data lake is **transformed into a highly structured** state that the Amazon **Redshift** data warehouse **can use**.
- The data engineer or data scientist would need to structure, enrich, and validate the data before the warehouse can use it. Data is stored in different zones as these steps are performed.



- Data structuring prepares the data for the trusted zone. Preparation for this zone might include activities such as conforming to the schema and partitioning specifications for the dataset based on data warehouse tables. Then, data enriching is performed on data in the trusted zone, modeling the data so that it can be joined with other datasets and stored in the curated layer. Data validation would also occur as part of these transformations before storing data into the next zone.
- Datasets in the **curated layer** are ready for the Amazon **Redshift** data warehouse to ingest to make them available for **low-latency access** and **complex SQL querying**.
- Depending on the use case, **additional discovery** might occur before the structuring, enriching, and validating steps take place. Additional cleaning might be needed as well. **The process is often iterative.**
- Data wrangling steps include discovery, structuring, cleaning, enriching, validating, and publishing.

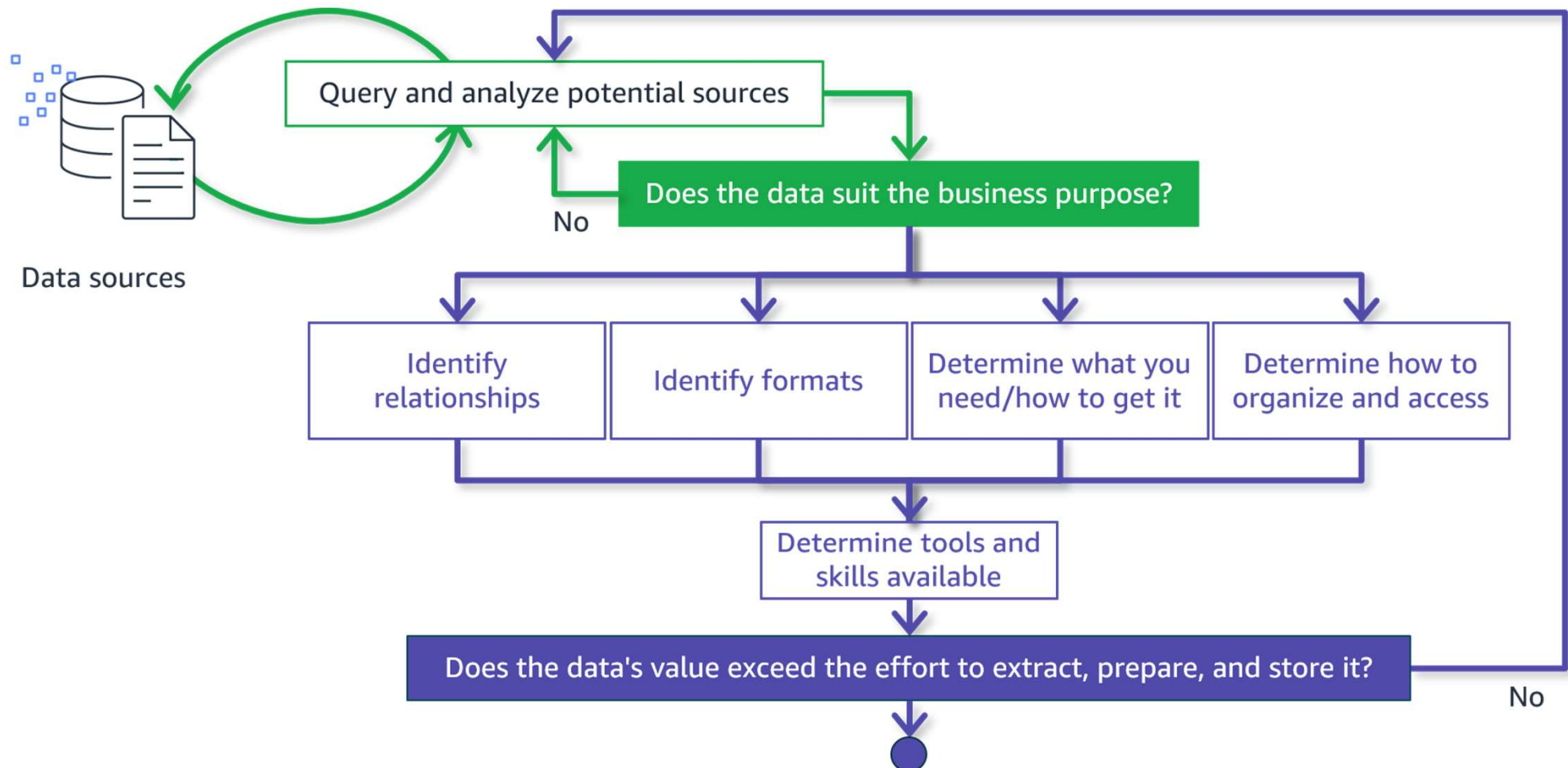
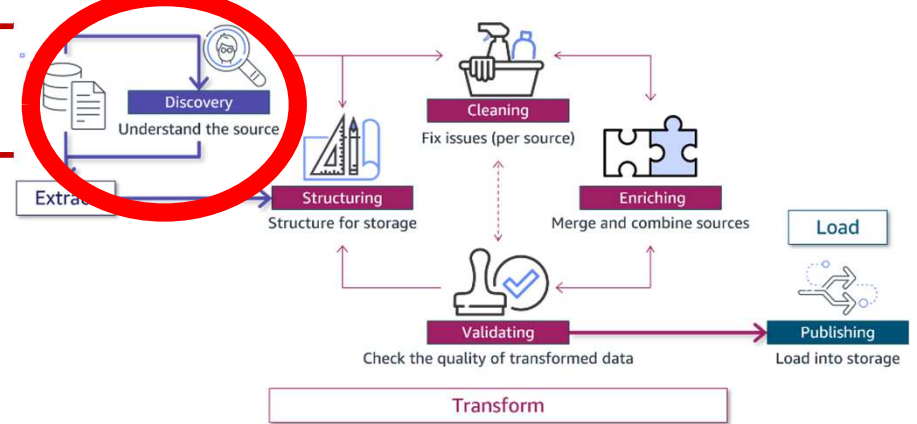
Key takeaways: Data wrangling introduction



- Data wrangling describes the multi-step process of transforming large amounts of unstructured data or sets of structured data from multiple sources for an analytics use case.
- Data wrangling is especially important for data scientists when building ML models.
- Data wrangling steps might overlap, be iterative, or not occur at all in some ingestion processes.
- Data wrangling steps include discovery, structuring, cleaning, enriching, validating, and publishing.

ETL & ELT

Data discovery





- The data discovery step is iterative
- As a data engineer, you need to find sources that you think might be useful, query them, and analyze the raw data to decide whether the sources have value for your business purpose.
- Key tasks of this step include the following: Decide if a source seems to serve your business purpose. If it does, then you want to dig deeper into the data to do the following:
 - Identify what relationships exist within and between the raw data sources
 - Identify the data formats (CSV, Parquet, ORC,...).
 - Determine the range of data that you want from each source
 - Determine how you should organize the data to make it easy—but secure—to access and cost-effective to ingest and store. (For example, you might use the zone approach, which was illustrated earlier, to segment data that comes into the data lake into "raw," "landing," and "trusted" zones.)
- Based on your analysis of the data relationships, formats, and organizational requirements, determine what tools you need.
- Based on these discovery activities, you might decide to work with the source. Or, you might determine that the effort and cost to extract and prepare the data for the intended business purpose doesn't provide enough value. If you determine that it's not worth the effort, you might start the whole process again and



- A company with a software as a service (SaaS) product recently acquired a startup company.
- The startup company's customer support team uses a different ticketing system than the company that acquired them.
- A data engineer has been asked by a data analyst to ingest support ticket data from the two products to perform an initial exploration of relationships between support experiences, ticket volumes, and contract renewals.
- The analyst wants to know the number and types of technical issues that have been worked on or closed for each customer in 2020. They also want each regional sales team to have access to only their customers.
- The existing sales analytics pipeline is built on AWS services and currently houses customer data in an Amazon Redshift data warehouse. However, it doesn't include customer support data.

Data discovery example: Query data sources

Ticket table in customer support system (supp1)

ticket_id	requestor_id	submitter	assignee	group	subject	status	priority	ticket_type	create_date	updated_date	solved_date
							High				
							Low				

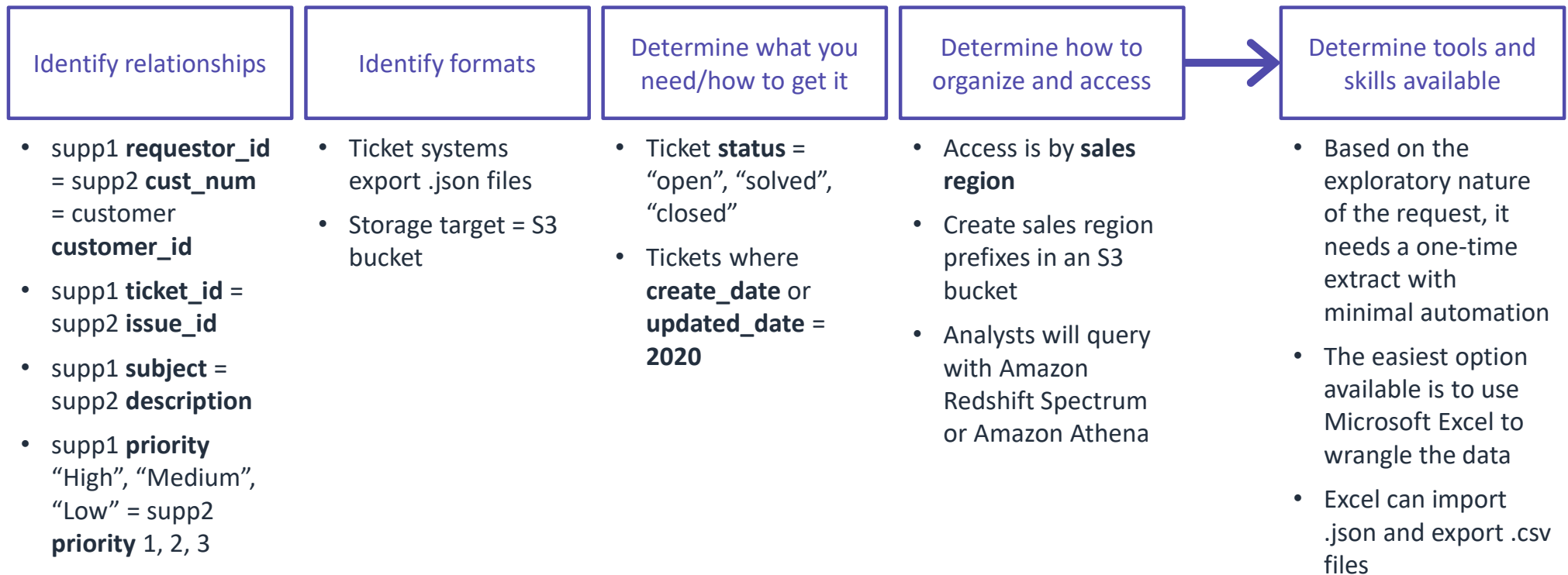
Ticket table in the acquired customer support system (supp2)

issue_id	cust_num	description	status	priority	create_date	updated_date	closed_date
				1			
				3			

Customer table in the data warehouse

customer_id	cust_name	primary_poc	status	sales_group

Data discovery tasks: Example scenario



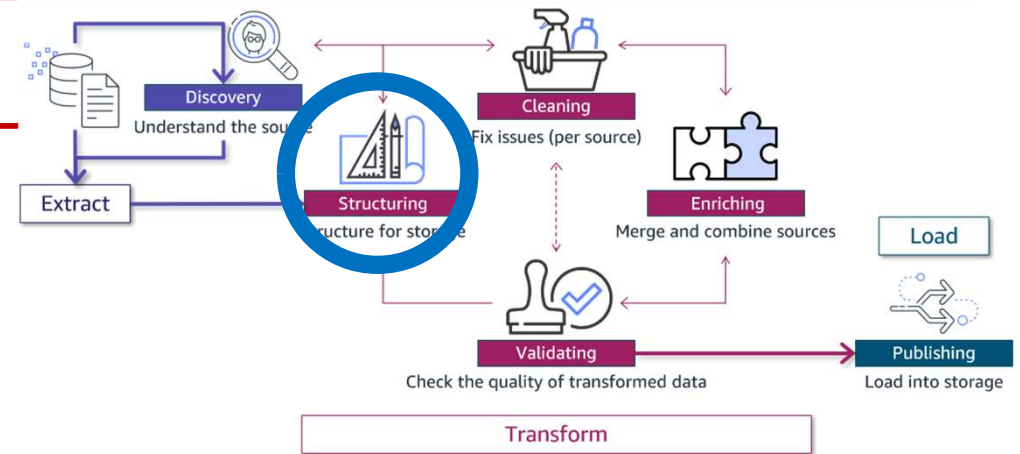
Key takeaways: Data discovery



- Data discovery is iterative and might have a different focus depending on your role.
- Tasks performed during the data discovery step include the following:
 - Identifying relationships and formats
 - Determining how to filter the incoming data and how to organize it in your target storage
- The two primary outcomes of the data discovery phase are identification of the tools and resources required and a decision to move forward.

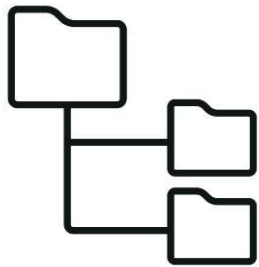
ETL & ELT

Data structuring



- Data structuring is about mapping data from the source file or files into a format that supports combining it with other data and storing it for its intended purpose.
- Key tasks in data structuring include the following:
 - Organize storage.
 - Parse the source file or files.
 - Map fields.
 - Manage file size.

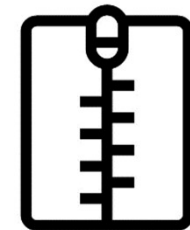
Decisions and tasks in data structuring



```
{  
  "ticket_id": 900864,  
  "requestor_id": 1800445,  
},
```



SALES				
CUST				



Organize storage

- Control access.
- Create the folder structure.
- Establish partitions.

Parse the source file

- Convert strings and patterns into fields or attributes.

Map fields

- Match source fields to target fields.

Manage file size

- Split or merge files.
- Compress files.

Data structuring example: Parse supp2 file and map fields

Import .json and let Excel parse the file.

Update column headings to map to supp1 and data warehouse field names.

A	B	C	D	E	F	G	H
issue_id ticket_id	cust_num customer_id	description subject	status	priority	create_date	updated_date	closed_date solved_date
900865		Error message 808 <*&#	Closed	2	6/26/20 2:45:29	6/27/20 21:31:48	6/27/20 23:30:09
900866	2020411	Nullpointer exception on save	Open	1	4/2/20 12:15	4/5/20 11:15	
900867	2022010	Question about grades report	Open	1	4/6/20 10:56		

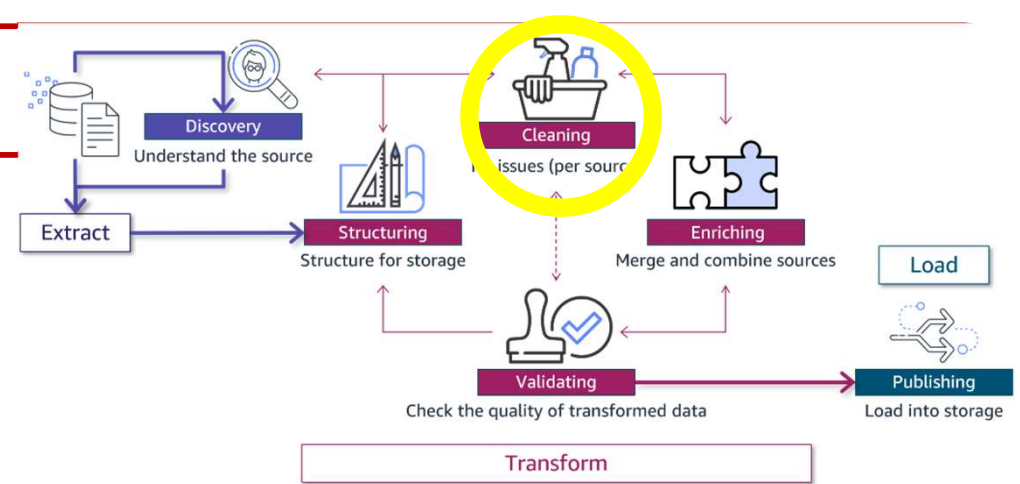
Key takeaways: Data structuring



- Data structuring is about mapping raw data from the source file or files into a format that supports combining it and storing it with other data.
- Data structuring includes organizing storage and access, parsing the source file, and mapping the source file to the target.
- Data structuring also includes strategies and methods to optimize file size, such as splitting or compressing files.

ETL & ELT

Data cleaning



- Data cleaning includes tasks such as the following:
 - Remove unwanted data.
 - Fill in missing data values.
 - Validate or modify data types.
 - Fix outliers.
- Depending on **who does the cleaning**, the methods and results might be different. For example, a data engineer who is only responsible for successfully loading the data into the pipeline might make sure that blank fields are replaced with values that fit the data type. However, a data analyst might replace blanks by adding corrected values.

Decisions and tasks in cleaning



001	42	4/2	abc
002		4/5	abc
003	78	5/5	
004	14	5/8	def
005	4200	5/9	def

Remove data

- Remove unwanted columns.
- Remove duplicate values.
- Remove "garbage" values.

001	42	4/2
002	0	4/5
003	78	5/5
004	14	5/8
005	4200	5/9

Fill missing data

- Fill null columns.
- Fill missing values for mandatory fields.



001	42	22-04-02
002	0	22-04-05
003	78	22-05-05
004	14	22-05-08
005	4200	22-05-09

Check data types

- Validate or modify data types.

001	42	22-04-02
002	0	22-04-05
003	78	22-05-05
004	14	22-05-08
005	42	22-05-09

Fix outliers

- Identify outliers, and either remove or fix data as appropriate.

Data cleaning example: Remove and fix data

Replace blanks in the required **customer_id** field with a placeholder.

Remove invalid characters from the **subject** field.

Replace the numeric **priority** field with the text field equivalent of supp1.

Modify the timestamps to conform to the data warehouse date format.

A	B	C	D	E	F	G	H
ticket_id	customer_id	subject	status	priority	create_date	updated_date	solved_date
900865	9999999	Error message 808 <*&#	Closed	Medium	2020-06-26 02:45:29	2020-06-27 21:31:48	2020-06-27 23:30:09
900866	2020411	Nullpointer exception on save	Open	High	2020-04-02 12:15:00	2020-04-05 11:15:00	
900867	2022010	Question about grades report	Open	High	2020-04-06 10:56:00		

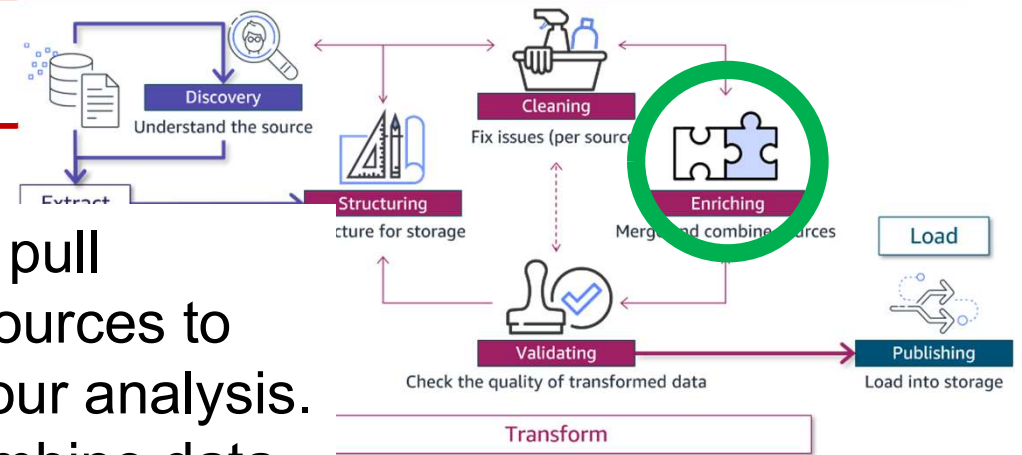
Key takeaways: Data cleaning



- Data cleaning is about preparing the source data for use in a pipeline.
- Cleaning is usually done for each source based on the characteristics of that source.
- Cleaning includes tasks such as removing unneeded, duplicate, or invalid data, and fixing missing values and data types.
- How the cleaning tasks resolve issues depends on the role of the person who cleans the data.

ETL & ELT

Data enriching



- In the data enriching step, you start to pull together related data from disparate sources to create the dataset that you need for your analysis. At this stage, you start to merge or combine data from different sources to support your use cases.
- 1) Data enriching could be relatively simple, such as merging similar files from multiple sources.
 - 2) This step could also be much more complex. Example: combining public health data, customer data from a healthcare provider's patient system and event data from a mobile app that monitors heart rate → This would require a much higher level of transformation and processing to merge these three very different sources into a dataset to be analyzed or acted upon.
 - 3) Enriching might also include aggregations or other calculations that are added as new fields to the dataset.

Data enriching example: Combine disparate sources

Combine rows from supp1 with rows from supp2.

Add a **ticket_type** category for supp2 rows.

Add the sales region.

A	B	C	D	E		F	G	H	
ticket_id	customer_id	subject	status	priority	ticket_type	create_date	updated_date	solved_date	sales_region
900865	9999999	Error message 808	Closed	Medium	Problem	2020-06-26 02:45:29	2020-06-27 21:31:48	2020-06-27 23:30:09	
900866	2020411	Nullpointer exception on save	Open	High	Problem	2020-04-02 12:15:00	2020-04-05 11:15:00		USEast
900867	2022010	Question about grades report	Open	High	Question	2020-04-06 10:56:00			USWest
900862	1744899	Nightly batch failing	Closed	Medium	Problem	2020-01-04 08:06:00	2020-04-31 09:00:00	2020-04-31 09:00:00	Europe
900865	1744005	Intermittent error message	Open	Low	Problem	2020-01-06 11:14:00			Europe

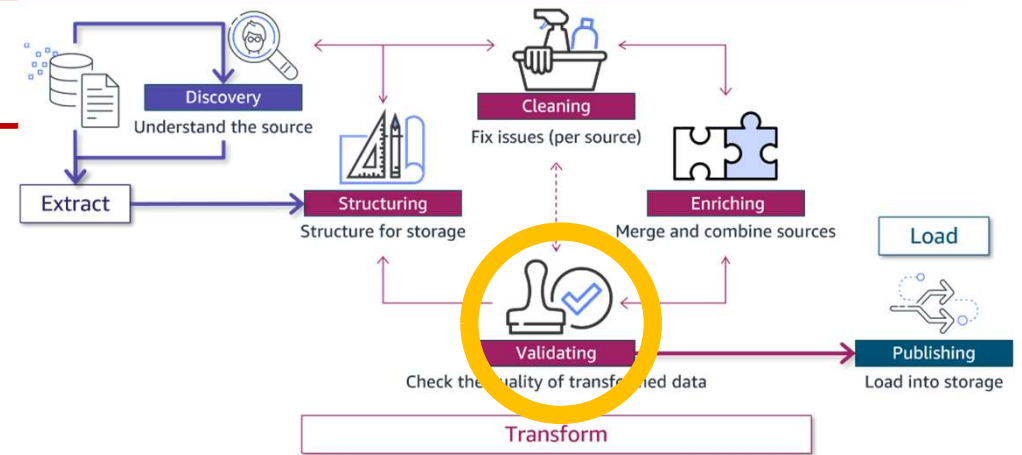
Key takeaways: Data enriching

- Data enriching is about combining your data sources together and adding value to the data.
- During enriching, you might merge sources, add additional fields, or calculate new values.



ETL & ELT

Data validating



- In the data validating step, you make sure that the dataset you have created for your analytics or ML use case has the veracity that you need.
- Validating has some similarities with the cleaning step. The initial cleaning step focuses on making sure that the data is suitable to successfully load into your pipeline. But the validating step is about ensuring the integrity of the dataset that you have created through the structuring, cleaning, and enriching steps for the combined source files.
- validating might include a combination of automated and manual checks, and could be performed by different roles.

Decisions and tasks in validating



Audit your work

- Count expected rows.
- Check consistency.
- Check expected formats and data types.
- Check for duplicates.
- Check for PII.
- Check for outliers.

Fix the data

- Address audit findings.
- Address issues at the source.
- Fix issues with integration tools or scripts.

Data validating example

Address duplicate ticket IDs.

Find the correct **customer_id**, fix in the source, and run a script to get **sales_region**.

A	B	C	D	E		F	G	H	
ticket_id	customer_id	subject	status	priority	ticket_type	create_date	updated_date	solved_date	sales_region
900865	2070555	Error message 808	Closed	Medium	Problem	2020-06-26 02:45:29	2020-06-27 21:31:48	2020-06-27 23:30:09	USEast
900866	2020411	Nullpointer exception on save	Open	High	Problem	2020-04-02 12:15:00	2020-04-05 11:15:00		USEast
900867	2022010	Question about grades report	Open	High	Question	2020-04-06 10:56:00			USWest
900862	1744899	Nightly batch failing	Closed	Medium	Problem	2020-01-04 08:06:00	2020-04-31 09:00:00	2020-04-31 09:00:00	Europe
900865	1744005	Intermittent error message	Open	Low	Problem	2020-01-06 11:14:00			Europe

Count total rows and check against rows from individual systems.

Supp2 rows()=3
Supp1 rows()=2
Target rows()=5

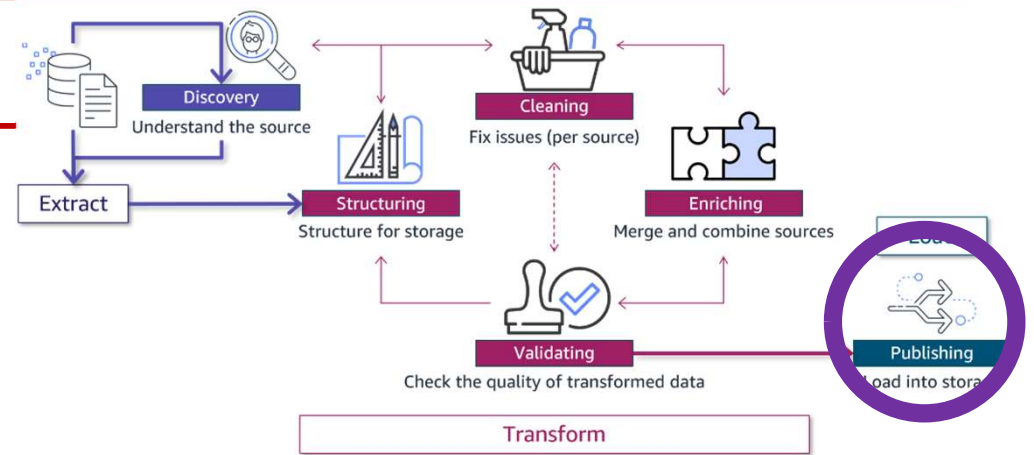
Key takeaways: Data validating



- Data validating is about ensuring the integrity of the dataset that you have created.
- Validating tasks might overlap with cleaning tasks.
- This step might be iterative as you check the results of your work, find issues, and address them.

ETL & ELT

Data publishing



- In the data publishing step, you make the dataset available for use within the pipeline.
- During publishing, you apply your structuring decisions. You need to write data to the appropriate folders or partitions, and apply the file management techniques that you decided on. For example, you might split files to make them load faster and compress them to save on storage costs in the data lake.
- As part of your publishing process, you also need to make sure that the storage locations that you have set up allow least-privilege access to the users or systems that should be able to consume the data. You also need to save metadata about the dataset to make it findable to consumers.

Decisions and tasks in publishing



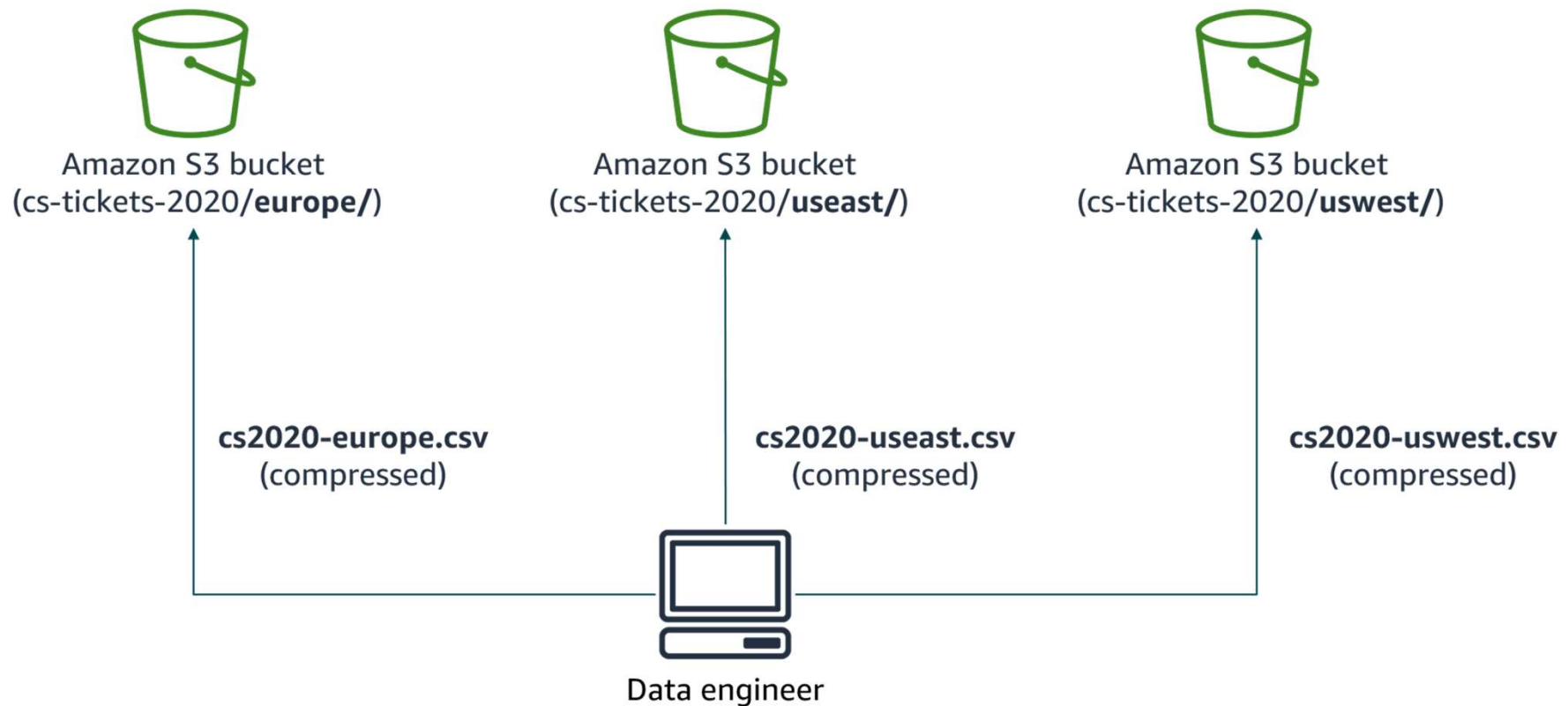
Move data into permanent storage

- Apply file management techniques (formats, compression, organization).
- Securely transfer data from source to destination.
- Write the dataset or datasets to the correct storage locations.

Make data available to consumers

- Apply appropriate access controls to data.
- Save metadata about the dataset.
- Configure how often and when to run the extract.
- Log the update history.

Data publication example: Moving data to permanent storage



Data publication example: Uploading the files

Export to **europa.csv**; zip as **europa.zip**; upload to **s3://cs-tickets-2020/europa/cs2020-europa.zip**

```
ticket_id,customer_id,subject,status,priority,ticket_type,create_date,updated_date,solved_date
,sales_region
900862,1744899,Nightly batch failing,Closed,Medium,Problem,2020-01-04 08:06:00,2020-04-31
09:00:00,2020-04-31 09:00:00,Europe
100865,1744005,Intermittent error message,Open,Low,Problem,2020-01-06 11:14:00,, ,Europe
```

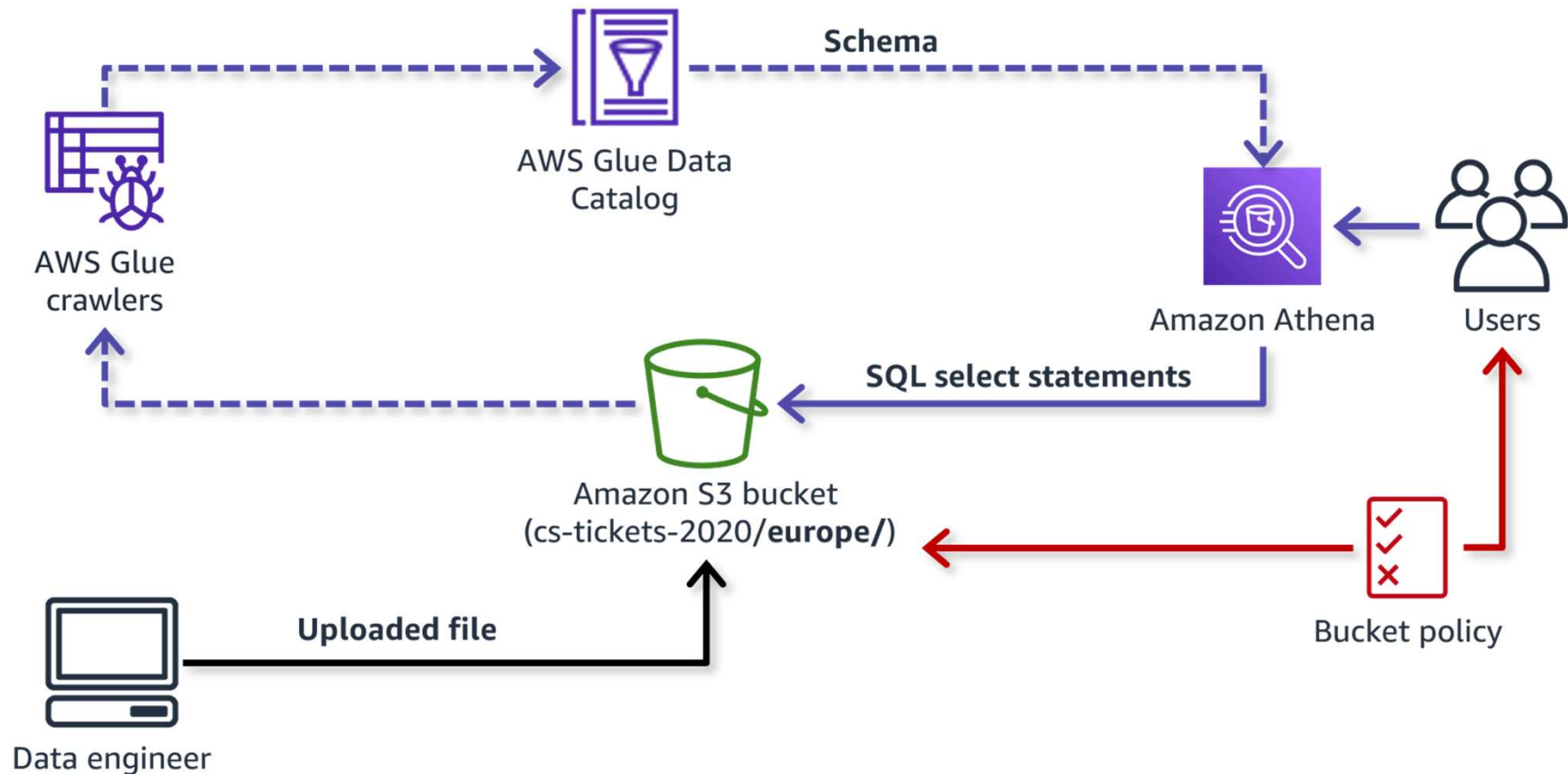
Export to **useast.csv**; zip as **useast.zip**; upload to **s3://cs-tickets-2020/useast/cs2020-useast.zip**

```
ticket_id,customer_id,subject,status,priority,ticket_type,create_date,updated_date,solved_date
,sales_region
900865,2070555,Error message 808,Closed,Medium,Problem,2020-06-26 02:45:29,2020-06-27
21:31:48,2020-06-27 23:30:09,USEast
900866,2020411,Nullpointer exception on save,Open,High,Problem,2020-04-02 12:15:00,2020-04-05
11:15:00,, ,USEast
```

Export to **uswest.csv**; zip as **uswest.zip**; upload to **s3://cs-tickets-2020/uswest/cs2020-uswest.zip**

```
ticket_id,customer_id,subject,status,priority,ticket_type,create_date,updated_date,solved_date
,sales_region
900867,2022010,Question about grades report,Open,High,Question,2020-04-06 10:56:00,, ,USwest
```

Data publication example: Making the data available



Key takeaways: Data publishing



- Data publishing is about making the dataset available for use.
- In this step, you move the data to the storage layer of the pipeline. The decisions that you make during the structuring step about organization and file management are used to load the data into storage.
- At this step, you also need to provide methods to update the dataset and monitor ongoing ingestion.

Annex: data silos



- Data silos are sets of data that are only accessible to one part of the company, isolated from the rest of the organization



- Data silos can arise for a number of reasons.
 - When an organization becomes **too large**, it becomes difficult to circulate data between departments. The more departments, offices and employees there are, the more complex communication becomes.
 - Similarly, when **growth is too rapid**, the company's structure is altered and the passage of data between the different stages of the hierarchy becomes tedious.
 - **Competition** can also arise between employees, with some teams deliberately refusing to transfer their data to prevent others from benefiting from it...
 - If companies **do not have the technologies and applications** they need to transfer information, it becomes difficult to circulate data.



- Data silos problems
 - Create a less collaborative environment within the company. Each team works independently, and is content with access to its own data. This divides the organization, prevents teams from collaborating on projects, and makes it impossible to share a common vision.
 - Data silos tend to slow down the business. Each team has to wait until they realize they need data that they don't have access to, then search for where that data is within the business, gain access to it manually, and finally analyze it for their own needs. As well as being a considerable waste of time, this process is so slow that the data may be obsolete by the time a team finally gains access to it.
 - Data silos are also a waste of storage space. If every employee who needs the same data stores it in his own department's folder, the information will be stored with unnecessary redundancy. This leads to unnecessary costs. It would therefore be more advantageous to make all the data available to the various departments from a centralized platform accessible to all employees within the company.
 - Each team may have access to its own version of the same data. Analyses will therefore be based on information that is no longer true, and may lead to the wrong decisions being taken.



- There are several solutions to combat the formation of Data Silos.
 - Choose a new platform to unify data management. Rather than letting each team use a different piece of software, it is preferable for all departments to use the same platform. By consolidating all the data in this way, it will be easier to share it between departments.
 - Cloud is a relevant solution, particularly for moderate volumes of data. Data can be stored in a single Cloud archive rather than in multiple silos. Backups ensure that data is integrated across all members and departments of the company.
 - Data lake & Data warehouses: repositories for all the data collected by a company. These systems are optimized for data access and analysis, and provide a 360-degree view.